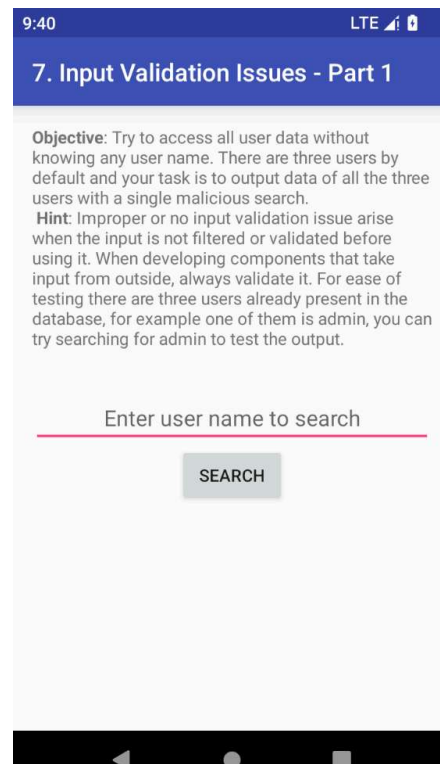


Challenge 7 Input Validation part 1



The screenshot shows a mobile application interface. At the top, a status bar displays '9:40' and 'LTE' with signal icons. Below this is a blue header bar with the text '7. Input Validation Issues - Part 1'. The main content area has a light gray background and contains the following text:

Objective: Try to access all user data without knowing any user name. There are three users by default and your task is to output data of all the three users with a single malicious search.

Hint: Improper or no input validation issue arise when the input is not filtered or validated before using it. When developing components that take input from outside, always validate it. For ease of testing there are three users already present in the database, for example one of them is admin, you can try searching for admin to test the output.

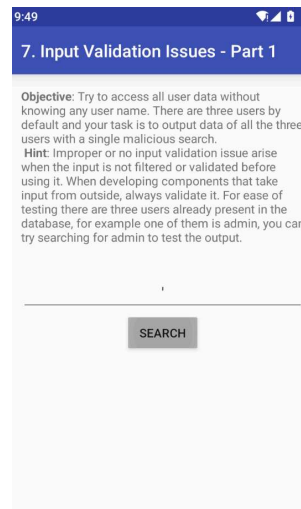
Below the text is a search input field with the placeholder text 'Enter user name to search'. A red underline is positioned below the input field. Below the input field is a gray button with the text 'SEARCH'.

Our next challenge is about Input Validation Issues

Input validation attack happens when the application could not sanitize the user input properly, and which could lead to data information leak, one of the simplest example of this is SQLi and XSS.

So in this challenge there are 3 default users, and if we could find any of them, we can see the secret data,

So let's try a simple sql injection, by sending standard single quote.



This screenshot shows the same mobile application interface as the previous one. The status bar now displays '9:49'. The header bar remains '7. Input Validation Issues - Part 1'. The objective and hint text are identical. In the search input field, a single quote character (') has been entered. The 'SEARCH' button is still present below the input field.

As we can see, there is an SQLite error in the logs, indicating that the application is potentially vulnerable to SQL injection.

Let's inject our payload to retrieve the list of users. By default, there are three users, and our task is to retrieve all user data with a single malicious payload.

Use the following payload: **'or '1'='1'--**

7. Input Validation Issues - Part 1

Objective: Try to access all user data without knowing any user name. There are three users by default and your task is to output data of all the three users with a single malicious search.

Hint: Improper or no input validation issue arise when the input is not filtered or validated before using it. When developing components that take input from outside, always validate it. For ease of testing there are three users already present in the database, for example one of them is admin, you can try searching for admin to test the output.

'OR '1'='1' --

SEARCH

User: (admin) pass: (passwd123) Credit card: (1234567812345678)
User: (diva) pass: (p@ssword) Credit card: (111122223334444)
User: (john) pass: (password123) Credit card: (5555666677778888)

We were successfully able to retrieve all user data with a single malicious query. Mission accomplished!

Challenge 8 Input Validation part 2

8. Input Validation Issues - Part 2

Objective: Try accessing any sensitive information apart from a web URL.
Hint: Improper or no input validation issue arise when the input is not filtered or validated before using it. When developing components that take input from outside, always validate it.

Enter the URL to view

VIEW

In this challenge we have to try accessing any sensitive information inside the device.

As we did in the previous challenge, we have a **.uinfo.txt** file in sdcard. Let's try accessing that file.

It is located in **file:///sdcard/.uinfo.txt**

10:00

8. Input Validation Issues - Part 2

Objective: Try accessing any sensitive information apart from a web URL.
Hint: Improper or no input validation issue arise when the input is not filtered or validated before using it. When developing components that take input from outside, always validate it.

file:///sdcard/.uinfo.txt

VIEW

test4:123456